

Fortran Interoperability: ISO_C_BINDING vs. Windows Named Pipes + JSON-RPC 2.0

Revision 0, 06/21/26

Introduction

There are two approaches that represent fundamentally different philosophies for the transfer of data between Fortran and other languages: **ISO_C_BINDING** is a *tightly-coupled, in-process* interoperability mechanism, while **Named Pipes + JSON-RPC 2.0** is a *loosely-coupled, inter-process* communication protocol. They are not direct competitors — each is the right tool in different scenarios — but they can overlap in purpose when the goal is simply "get data from Fortran into another language."

Below is a quick summary of the ISO_C_BINDING and JSON RPC 2.0 methodologies as applied to transferring data in and out of a Fortran program. Note that this summary is being provided to educate the reader as to the advantages and disadvantages of each method; neither method is "the best" for all applications.

ISO_C_BINDING

What It Is

ISO_C_BINDING is an intrinsic Fortran module (standardized in Fortran 2003) that allows Fortran code to share memory, call functions, and expose subroutines directly with C and .NET languages. Because virtually every modern language (VB.NET, C#.NET, F#.NET, Python via ctypes/cffi, C++, Rust via FFI, Julia, R via .C()/Fortran()) can interoperate with C, ISO_C_BINDING effectively gives Fortran a universal interop bridge.

Advantages

- 1. Performance** This is the dominant advantage. Data is shared in-process, often in the same memory space with zero copying. Passing a 1-million-element array from Fortran to C++ or Python (via ctypes) costs nothing beyond a pointer dereference. For HPC workloads — numerical solvers, fluid dynamics, climate models — this is very desirable.
- 2. Standardized and Portable** ISO_C_BINDING is part of the Fortran standard. It works on Linux, macOS, Windows, and exotic HPC platforms (Cray, IBM). The code is written is not tied to a specific OS or IPC mechanism.
- 3. Type Safety at the Boundary** The module provides named kind parameters (C_INT, C_DOUBLE, C_FLOAT, etc.) that map exactly to C's type sizes. The compiler can catch mismatches. This is much safer than the old pre-standard approach of relying on assumptions.
- 4. Direct Array and Pointer Sharing** Using C_LOC, C_F_POINTER, and C_F_PROCPINTER, one can pass raw Fortran arrays — including multi-dimensional arrays — directly to C code. The receiving side gets a real pointer to real memory. No serialization, no copies.

5. Mature Ecosystem Decades of tooling, documentation, and community knowledge exist. Libraries like LAPACK and BLAS expose C interfaces via BIND(C) routinely. Python's numpy, scipy, and ctypes have well-documented patterns for calling Fortran via this route.

6. Zero Runtime Dependencies There is no server, no broker, no protocol library. The interop is resolved at link time. Deployment is simple.

7. Fortran 2018 Extensions (CFI_cdesc_t) The 2018 standard added the C descriptor interface, allowing C code to receive and manipulate assumed-shape Fortran arrays without losing rank and bounds information. This closes a long-standing gap.

Disadvantages

1. Language Must Be Able to Call C Foreign Function Interface (FFI) The "other language" must support calling C foreign functions. Most languages do (Python, C++, Rust, Julia, R), but some don't (pure Java without JNI, JavaScript in a browser, languages sandboxed from native calls). Each language also has its own FFI syntax and quirks.

2. Column-Major vs. Row-Major Memory Layout Fortran stores multi-dimensional arrays in column-major order; C, Python (numpy by default), and most other languages use row-major. Passing a 2D array directly gives the other side data in the wrong order unless one explicitly transposes or uses C_F_POINTER with adjusted strides. This is a frequent source of subtle bugs.

3. String Handling is Painful Fortran strings are fixed-length, space-padded, and not null-terminated. C strings are null-terminated. CHARACTER(KIND=C_CHAR) bridging works but requires explicit length handling on both sides. There is no clean "pass a Fortran string to Python" equivalent without wrapper code.

4. Tight Coupling and Application Binary Interface (ABI) Fragility Both sides must be compiled and linked together (or against a shared library). Changing a function signature in Fortran requires recompiling the consuming code. If one has a Python script calling a Fortran .so, any incompatible change in the library breaks the script silently at runtime.

5. Complexity for Complex Data Structures Passing scalars and arrays is straightforward. Passing structs with pointers-to-pointers, variable-length arrays, or polymorphic types requires careful BIND(C) type definitions and is error-prone. There is no automatic marshalling of complex object graphs.

6. No Language-Version Isolation If the Fortran code uses a Fortran 2003 feature and is compiled with gfortran 4.8, and the C caller was compiled with a different compiler on the same platform, subtle ABI differences (name mangling, alignment) can cause silent failures. Debugging these is notoriously difficult.

7. Windows-Specific Friction On Windows, name mangling differs between compilers (Intel Fortran vs. gfortran vs. MSVC). DLL export conventions add another layer. ISO_C_BINDING with BIND(C) resolves most of this, but getting a Fortran DLL to load correctly in Python on Windows is still far more work than on Linux.

8. Single-Process Only (Normally) Both the Fortran code and the consuming code must run in the same process (or one must load the other as a shared library). One cannot, for instance, run Fortran on a GPU node and Python on a CPU node and share data this way without additional infrastructure.

Windows Named Pipes + JSON-RPC 2.0

What It Is

Named pipes are a Windows IPC mechanism that provides a bidirectional byte stream between processes using a file-like interface. The Fortran process opens or creates a named pipe (e.g., `\\.\pipe\my_fortran_server`), and the other-language process connects to it. Both sides read and write byte streams.

JSON-RPC 2.0 is a lightweight (?), stateless remote procedure call protocol encoded in JSON. A caller sends:

```
{"jsonrpc": "2.0", "method": "solve", "params": {"matrix": [...], "rhs": [...]}, "id": 1}
```

And the Fortran side responds:

```
{"jsonrpc": "2.0", "result": {"solution": [...]}, "id": 1}
```

Together, they create a language-agnostic RPC system where Fortran acts as a server (or client) and any language that can open a pipe and parse JSON acts as a peer.

Advantages

1. Complete Language Agnosticism Any language that can open a named pipe and serialize/deserialize JSON can communicate with the Fortran process. JavaScript, Java, Go, Kotlin, PHP, Ruby — none of them need a C FFI. This is a much wider compatibility than ISO_C_BINDING.

2. Process Isolation and Fault Tolerance The Fortran process and the consumer are separate OS processes. If the Fortran program crashes, it does not take down the calling process. The consumer can detect the broken pipe and restart the Fortran process. This is architecturally robust in ways that in-process FFI cannot be.

3. Independent Compilation and Deployment The Fortran binary and the consumer can be compiled, updated, and deployed independently. As long as the JSON-RPC method signatures don't change, both sides evolve independently. This enables semantic versioning of the Fortran "service."

4. Natural Fit for LSP-Style Architectures The Language Server Protocol (LSP), used by VS Code and most modern editors, is built on JSON-RPC 2.0 over stdin/stdout or named pipes. If the Fortran code is a computational backend serving a development tool, analysis service, or IDE plugin, this is already the standard.

5. Debuggability JSON is human-readable. One can log every request and response, inspect them, replay them, and build test suites around captured traffic. Debugging a binary FFI call that corrupts memory is far harder than reading a JSON log.

6. Concurrency and Multiplexing (JSON-RPC) JSON-RPC 2.0 supports batched requests and asynchronous notifications. Multiple consumers can connect to a named pipe server simultaneously (with proper server-side handling). One can pipeline requests without waiting for each response.

7. No Shared Memory Concerns Each side owns its own memory. There are no column-major/row-major issues, no pointer aliasing bugs, no C_LOC misuse. The data is serialized to text, transmitted, and deserialized independently on each side.

8. Cross-Machine Extensibility Windows named pipes can provide inter-process communications between two PCs on the same network; switching to TCP sockets uses the same JSON-RPC protocol with minimal code changes. The same Fortran solver can later be exposed as a network service without architectural change.

9. Suitable for Long-Running Services Fortran numerical solvers often take seconds to hours. Named pipes + JSON-RPC naturally model an async request-response pattern where the consumer sends a job, waits on the pipe, and receives results when ready — much more natural than blocking an FFI call.

Disadvantages

1. Serialization Overhead JSON is text-based. Serializing a 1-million-element REAL(8) array to JSON produces roughly 15–20 MB of ASCII text. Parsing it on the other side takes measurable time. For large numerical datasets, this overhead is often prohibitive.

2. Precision Loss IEEE 754 double-precision floats have 15–17 significant decimal digits. JSON has no binary float encoding — numbers are written as decimal strings. Round-tripping through JSON (1.0000000000000002 → text → parse) can lose the last 1–2 ULPs. For most applications this is negligible, but for numerically sensitive Fortran HPC code, it can matter.

3. Windows-Only (Named Pipes) Windows named pipes (\\.\pipe\name) do not exist on Linux or macOS. POSIX named pipes (mkfifo) are unidirectional and have different semantics. A Fortran codebase that runs on Linux HPC clusters and Windows workstations would need two different IPC mechanisms or a fallback to TCP sockets. This significantly undermines portability.

4. Latency Every call crosses a process boundary, goes through the OS pipe buffer, and involves context switches. For fine-grained interop (calling a Fortran function thousands of times per second in a tight loop), this latency may be detrimental. ISO_C_BINDING has essentially zero per-call overhead; named pipes add microseconds to milliseconds per round-trip.

5. Fortran JSON Libraries Are Immature There is no “official” standard JSON library in Fortran, although the intent of this project is to provide a ‘proof of concept’ version that can be built upon by the community.

6. Complexity of the Server Loop The Fortran code must implement a read-parse-dispatch-serialize-write event loop. This is non-trivial in Fortran, which has no native async I/O, no event loop primitives, and limited string handling. The server infrastructure itself may be a maintenance burden.

7. No Native Fortran Array Type in JSON JSON arrays are untyped. [1.0, 2.0, 3.0] could be floats or integers. Multi-dimensional arrays must be flattened (losing shape information) or represented as nested arrays (expensive to parse). Shape, rank, and type must be transmitted out-of-band in the JSON structure, adding protocol complexity.

8. Security Surface Named pipes on Windows have an ACL-based security model. If the pipe is not properly secured, other processes on the same machine can connect to it. For a local solver this is usually low risk, but it is a non-issue with in-process FFI.

Side-by-Side Summary

Criterion	ISO_C_BINDING	Named Pipes + JSON-RPC 2.0
Performance	Excellent (zero-copy, in-process)	Poor for large data (serialization cost)
Language compatibility	Languages with C FFI only	Any language with pipe + JSON support
OS portability	Excellent (standard)	Windows-only for named pipes
Fault isolation	None (shared process crash)	Full (separate processes)
Deployment coupling	Tight (shared library versioning)	Loose (independent binaries)
Debuggability	Hard (binary, memory-level)	Easy (human-readable JSON logs)
Float precision	Perfect (native binary)	Minor risk of rounding (text encoding)
Array interop	Native, but layout mismatch risk	Requires shape/type metadata in protocol
String handling	Awkward	Natural (JSON strings)
Complex data structures	Painful	Easy (JSON objects)
Long-running async jobs	Unnatural (blocking call)	Natural (async request-response)
Fortran library ecosystem	Mature	Immature (json-fortran is good, but limited)
Setup complexity	Moderate	High (server loop, protocol, library)

When to Use Which

Use ISO_C_BINDING when:

- Performance and data throughput are paramount (HPC, numerical solvers, tight loops)
- The other language supports C FFI (C++, Python, Rust, Julia, R)
- Distributing a reusable library

- The data is primarily arrays of numeric types

Use Named Pipes + JSON-RPC 2.0 when:

- The other language cannot easily call C FFI (Java, JavaScript, Go in some contexts)
- Process isolation and independent deployment matter more than throughput
- Building a long-running service or LSP-style tool
- The data is structured/heterogeneous (mixed types, variable-length records, metadata-rich)
- Debuggability and protocol transparency are priorities
- Building something that may later be exposed over a network